

Sistemi UAV:

UAV (Unmanned Aerial Vehicle): Aeromobile senza pilota a bordo.

Può essere pilotato da remoto tramite:

Ground Control Station (GCS) → la stazione di terra con cui l'operatore controlla il drone.

Autopilota → dispositivo di bordo che esegue le istruzioni o segue traiettorie preimpostate.

Ground Control Station (GCS):

Sistema di terra che permette il controllo remoto.

Funzioni:

- Pilotaggio e modifica dell'assetto in tempo reale.
- Controllo parametri di volo e dati dai sensori (es. immagini, misurazioni).

Autopilota:

Dispositivo elettronico che controlla i motori e gli attuatori.

Funzioni:

- Esegue le direttive ricevute dalla GCS.
- Mantiene l'assetto sicuro durante il volo.

Classificazione, utilizzi e sensori:

Classificazione UAV (parametri principali):

- Raggio operativo (quanto lontano può volare).
- Quota di volo (altezza massima).
- Durata del volo (autonomia).
- Peso massimo al decollo.

Ambiti di utilizzo:

- Militare.
- Agricoltura (monitoraggio coltivazioni).
- Vigilanza e sicurezza.
- Riprese fotografiche/cinematografiche.
- Missioni di ricerca.

Affidabilità in volo → garantita dai sensori di bordo:

- Posizione e orientamento: accelerometro, giroscopio, magnetometro.
- Telerilevamento: GPS.
- Ambientali: temperatura, umidità, barometro, qualità dell'aria.

RECAP:

L'UAV è il drone.

La GCS comanda da terra.

L'autopilota stabilizza e controlla il volo.

La classificazione dipende da autonomia, quota, raggio e peso.

Viene usato in tanti settori (militare, civile, scientifico).

I sensori a bordo garantiscono stabilità e sicurezza.



Autopilota PX4:

Definizione generale di autopilota:

Insieme di routine software che governano un UAV senza bisogno dell'intervento diretto dell'uomo.

Permette al drone di navigare e mantenere stabilità in modo autonomo.

PX4: software open-source di controllo UAV

Licenza BSD → il software è libero e modificabile, ma non obbliga a rendere pubbliche le modifiche.
(Differenza rispetto a licenze copyleft come quella di Linux)

Sviluppo iniziato nel 2009 → comunità attiva e in continua evoluzione.

Modulare e flessibile, usato in molti droni commerciali e di ricerca.

Architettura PX4 → suddivisa in due layer principali:

1. Flight stack:

Moduli software che controllano direttamente l'hardware.

Implementa algoritmi di navigazione e controllo aeromobili.

Supporta diversi tipi di UAV:

Fixed-wing (ala fissa), VTOL (Vertical Take-Off and Landing), Multirotore (es. quadricotteri).

2. *Middleware:*

Si occupa di comunicazione e integrazione hardware.

Funziona come “ponte” tra il flight stack e i sistemi esterni (es. Ground Control Station, sensori, payload).

PX4 è un sistema operativo di volo open-source per UAV, molto diffuso perché:

- È flessibile e modulare.
- Supporta diversi tipi di droni.
- Si integra bene con hardware e sistemi di terra.
- È open-source, quindi adattabile a varie applicazioni (commerciali, militari, ricerca).

Architettura Flight Stack:

Componenti principali:

Estimator:

Riceve i dati dai sensori di bordo e li elabora per stimare lo stato attuale del velivolo.

Esempi:

- Assetto → da dispositivi inerziali (accelerometro, giroscopio).
- Altitudine → da barometro.

In pratica: “Traduce i dati grezzi dei sensori in informazioni utili per il controllo”.

Controller:

Usa lo stato stimato dall’estimator per calcolare le correzioni necessarie a raggiungere un setpoint (obiettivo).

Esempio: position controller → regola assetto e spinta per portare il drone nella posizione desiderata.

In pratica: “Decide cosa fare per mantenere o raggiungere la traiettoria”.

Mixer:

Traduce i comandi dei controller in segnali ai motori/attuatori, assicurandosi che non vengano superati i limiti strutturali.

Esempio: distribuisce la potenza sui vari rotori di un quadricottero per eseguire una manovra.

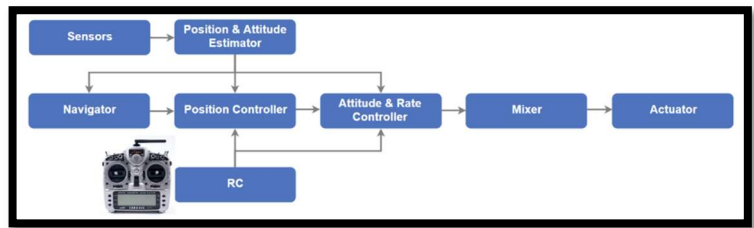
In pratica: “Trasforma i calcoli in comandi fisici”.

Riassumendo:

Estimator = interpreta i sensori.

Controller = decide le correzioni.

Mixer = traduce in comandi ai motori.



Architettura: middleware:

Componenti principali:

Driver per i sensori integrati a bordo:

Permettono di leggere i dati dai sensori e passarli al flight stack.

Esempio: accelerometro, giroscopio, GPS.

Moduli per la comunicazione:

uORB:

API per lo scambio di messaggi interni tra thread o processi.

Usa il paradigma publish/subscribe asincrono (un modulo pubblica i dati, altri si iscrivono per riceverli).

Serve a collegare i vari blocchi software dentro PX4.

MAVLink:

Protocollo leggero di comunicazione esterna.

Permette lo scambio di messaggi tra:

- UAV ↔ Ground Control Station (GCS).
- Autopilota ↔ altri componenti esterni.

Esempio: invio telemetria, comandi di missione, log.

Layer di simulazione:

Consente di testare i moduli software senza usare un vero UAV.

Simula sensori, ambiente di volo e attuatori.

È gestito dal sistema operativo in tempo reale (RTOS).

Architettura a strati:

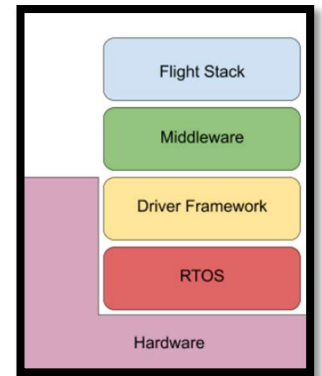
Flight Stack → algoritmi di controllo.

Middleware → comunicazione e astrazione dai sensori/attuatori.

Driver Framework → gestione diretta dei dispositivi hardware.

RTOS → sistema operativo real-time che gestisce task e tempi di esecuzione.

Hardware → sensori, attuatori, MCU.



Riassumendo:

Il middleware è la “colla” che:

Gestisce i driver dei sensori.

Fornisce la comunicazione interna (uORB) ed esterna (MAVLink).

Permette la simulazione senza hardware reale.

Caricamento del firmware:

Repository GitHub: Il codice sorgente del firmware PX4 è disponibile su GitHub: PX4-Autopilot

Generazione degli eseguibili → cross-compilazione:

Si usa il tool make.

I makefile definiscono quali sorgenti includere, librerie da linkare, regole per costruire i moduli caricabili.

Esempio:

```
make px4_fmu-v2_default
```

px4_fmu-v2 → target di compilazione, indica la versione di hardware su cui girerà il firmware.

default → configurazione scelta di default.

Caricamento sulla piattaforma:

Una volta compilato, il firmware si carica sull’hardware con: *make px4_fmu-v2_default upload*

In pratica:

Cross-compilazione = compilo su PC (x86) per una MCU (ARM).

Upload = trasferisco il binario sul controllore di volo.

Recap:

Il codice sorgente è su GitHub.

Si compila tramite makefile in cross-compilazione.

Con make ... upload si carica il firmware sulla scheda di controllo (es. PX4 FMU).

Integrazione di moduli esterni:

Firmware PX4 estensibile:

PX4 è modulare → permette di aggiungere nuovi moduli software oltre a quelli di base.

Può anche essere integrato con framework esterni (es. ROS).

Includere applicazioni esterne:

Le applicazioni aggiuntive si inseriscono in: *src/examples*

Per compilare queste applicazioni bisogna:

- Modificare il file *default.cmake* (in */boards/px4/fmu-v2*) → qui si indicano i moduli da includere.
- Aggiornare il file *CMakeLists.txt* con le direttive per compilare il nuovo codice.

Intelligenza e automazione:

È possibile rendere l'autopilota "intelligente" includendo:

- Sistemi di ragionamento automatico basati su una base di conoscenza.
- Algoritmi di machine learning per decisioni autonome.

N.B) "Machine learning base per via delle limitazioni hardware".

Recap:

PX4 è estensibile e può integrarsi con altri sistemi.

Si possono aggiungere moduli personalizzati tramite i file di configurazione della compilazione.

È possibile implementare algoritmi intelligenti (es. ML, AI), anche se le capacità sono limitate dall'hardware a bordo.

Software di simulazione:

Gazebo:

Ambiente 3D di simulazione per robot e UAV.

Offre una GUI completa per personalizzare e controllare scenari.

Permette di creare nuovi mondi o modelli (tramite XML).

Utile per testare algoritmi di:

- Computer vision (riconoscimento immagini).
- Object-avoidance (evitamento ostacoli).

Supporta simulazione multi-veicolo.

Spesso integrato con ROS (Robot Operating System).

N.B) È il simulatore più avanzato e realistico.

jMAVSim:

Basato su Java3D → implementazione grafica semplice.

Può essere controllato via ROS o QGroundControl (tramite UDP).

Limitazioni:

- Non permette di aggiungere sensori aggiuntivi o ostacoli.
- Nessuna ispezione visiva realistica.

Utile per test di base su dinamica di volo.

N.B) È il simulatore più leggero ma limitato, adatto a test rapidi.

QGroundControl:

Cos'è:

È un software che permette di configurare e guidare un velivolo dotato di autopilota PX4.

Funziona come Ground Control Station (GCS).

Funzionalità principali:

- Mission planning → pianificazione di voli autonomi.
- Visualizzazione mappe → mostra la posizione del drone in tempo reale.
- Gestione multi-veicolo → controllo contemporaneo di più UAV.
- Gestione sensori e dati → monitoraggio live dei parametri (batteria, GPS, assetto, altitudine, velocità).

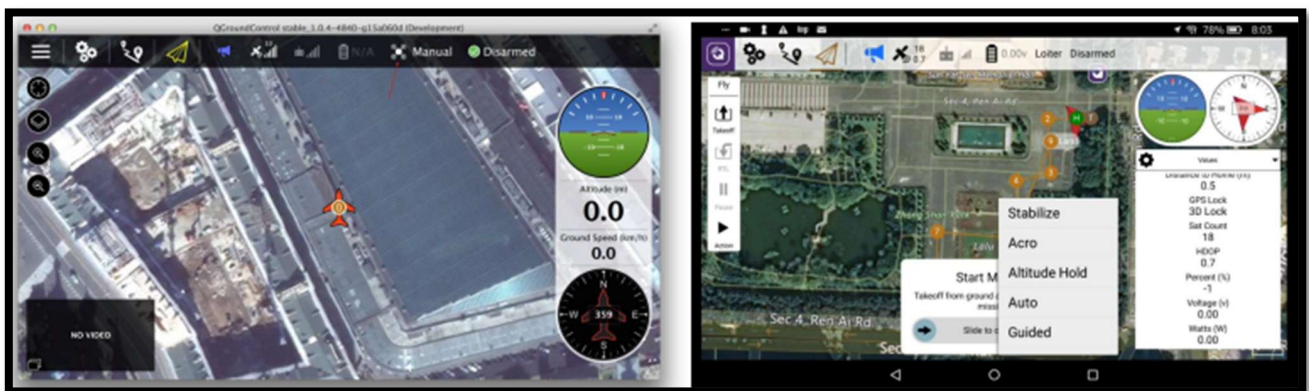
Piattaforme supportate

- Windows, macOS, Linux (PC).
- iOS, Android (mobile).

Questo lo rende molto flessibile, utilizzabile sia su PC che su tablet/smartphone durante le missioni.

QGroundControl è la **stazione di terra grafica** più usata per PX4, permette di:

1. configurare il drone,
2. pianificare missioni,
3. monitorare voli in tempo reale,
4. controllare più UAV da un'unica interfaccia.



Sistemi operativi real-time (RTOS):

Definizione:

Sono sistemi operativi specializzati per applicazioni che richiedono una risposta entro un intervallo di tempo preciso (*determinismo*).

Non conta solo la correttezza del risultato, ma anche che arrivi entro la deadline.

I tempi di esecuzione sono vincolati dalla certezza che il sistema riesca a rispondere in tempo.

Due tipi di sistemi real-time:

1. *Hard real-time*:

Le deadline non possono essere violate → anche un singolo ritardo è inaccettabile.

Usati in applicazioni critiche.

Esempi: Sistemi di controllo auto (airbag, ABS), Avionica, Controllo processi nucleari.

N.B) “Molto stringenti”.

2. *Soft real-time*

Le deadline possono essere violate occasionalmente, senza compromettere la sicurezza del sistema.

La violazione influisce solo sulla QoS (Quality of Service).

Esempi: Sistemi multimediali (streaming video/audio).

N.B) “Più permissivi”.

Esecuzione basata su eventi:

Il sistema risponde non appena si verifica un evento.

Garantisce che la risposta sia la più rapida possibile.

Un RTOS è deterministico: esegue operazioni entro tempi certi.

Hard real-time = critico, no violazioni.

Soft real-time = tollerante a piccoli ritardi.

Fondamentale per i droni, perché la stabilità dipende dal rispetto rigoroso delle deadline dei controlli di volo.

Scheduling negli RTOS:

1. Round-robin scheduling:

Algoritmo con preemption (interruzione di un processo per passare ad un altro).

Ogni processo riceve un time slice (quanto di tempo fisso per usare la CPU).

Semplice, ma può portare a violazione delle deadline se il time slice non è ben calibrato.

Adatto a sistemi non critici (Soft real time)

2. Priority-based scheduling

Ogni processo ha una priorità: quelli real-time hanno priorità più alta.

Funziona anche con preemption.

Non tiene conto direttamente delle deadline → per questo è usato in sistemi soft real-time.

Buono quando alcune deadline possono essere violate senza gravi conseguenze.

3. Rate-monotonic scheduling (RMS)

Algoritmo hard real-time.

Priorità statica: più un processo richiede la CPU frequentemente, più alta è la sua priorità.

Es: un processo che deve girare ogni 10 ms ha priorità più alta di uno che gira ogni 100 ms.

Basato su preemption.

Garantisce il rispetto delle deadline → rende il sistema deterministico.

Usato in RTOS come NuttX (quello usato da PX4).

In sintesi:

- Round-robin → equo ma non adatto a hard real-time.
- Priority-based → buono per soft real-time, ma non garantisce deadline.
- Rate-monotonic → ottimo per hard real-time, garantisce il rispetto delle deadline (usato da PX4 con NuttX).

Sistema operativo NuttX:

Caratteristiche generali:

RTOS (Real-Time Operating System) → rilasciato nel 2007.

Creato da Gregory Nutt.

Open-source sotto licenza BSD.

Scalabilità:

Progettato per funzionare su microcontrollori da 8 a 32 bit.

Perfetto per sistemi embedded con risorse limitate → ideale per i droni.

Linguaggio e API:

Scritto prevalentemente in C.

Basato su API standard POSIX e ANSI → compatibile con ambiente Unix/Linux.

Ampio uso delle API Unix:

- NuttShell (NSH): interprete comandi simile alla shell Linux.
- (Virtual) File System integrato.

Architettura modulare:

Kernel estensibile con moduli esterni.

Supporta protocolli di comunicazione per l'IoT.

Piattaforme supportate:

Compatibile con numerose architetture hardware: ARM, OpenRISC, Intel x8086, Atmel AVR, RISC-V, Zilog.

In sintesi:

NuttX è un RTOS leggero e modulare, compatibile con API standard Unix.

Usato come base software di PX4 perché:

- è deterministico.
- gira su microcontrollori con poche risorse.
- è scalabile e open-source.

NuttX in prodotti industriali:

Sony:

Registratori vocali Sony → usano NuttX come sistema operativo embedded.

Microcontrollori Sony Spresense → piattaforma di sviluppo per applicazioni IoT, basata su NuttX.

Motorola:

MotoMod → moduli aggiuntivi per smartphone Motorola.

NuttX gestisce la comunicazione tra smartphone e modulo esterno (es. fotocamera, proiettore).

Samsung:

TizenRT → variante di Tizen basata su kernel NuttX. (Sistema real-time)

Utilizzato in smart home e dispositivi IoT:

- Smart TV.
- Indossabili.
- Elettrodomestici intelligenti.



NuttX non è solo accademico, ma già usato da grandi aziende.

Settori di applicazione:

- Elettronica di consumo (Sony, Motorola).
- IoT e Smart Home (Samsung TizenRT).

Motivo del successo: scalabilità, leggerezza, real-time, open-source.

Architettura di NuttX:

User Layer (1° livello):

Funzione: comunicazione con l'utente.

Comprende:

- NuttX Shell (NSH) → interprete di comandi tipo Linux.
- Applicazioni e script esterni.
- Librerie POSIX → permettono alle applicazioni di comunicare con i meccanismi del sistema operativo.

Esempio: un programma sviluppato dall'utente utilizza le librerie POSIX per accedere ai servizi di NuttX.

OS Layer (2° livello):

Contiene la logica di base del sistema operativo:

- Algoritmi di scheduling.
- Gestione delle risorse.
- Meccanismi di IPC (Inter-Process Communication).

Include i driver hardware, divisi in:

- *Upper-half driver*: interfaccia lato software (comunicazione con il kernel).
- *Lower-half driver*: parte specifica che interagisce con l'hardware reale.

Qui si trovano anche moduli logici specifici della scheda (Board Specific Logic) per adattare il kernel all'hardware.

User layer: applicazioni, shell, librerie → per l'utente.

OS layer: kernel e driver → per gestire hardware e risorse.

Architettura modulare che rende NuttX scalabile e portabile.

Upper-half driver:

Espongono interfacce indipendenti dall'hardware allo user layer.

Implementano operazioni generiche (es. read, write).

Non conoscono i dettagli dell'hardware → per interagire con esso invocano i lower-half driver.

Rappresentano la parte astratta dei driver.

Lower-half driver:

Implementazione dipendente dall'hardware.

Contengono il codice specifico per pilotare un particolare dispositivo.

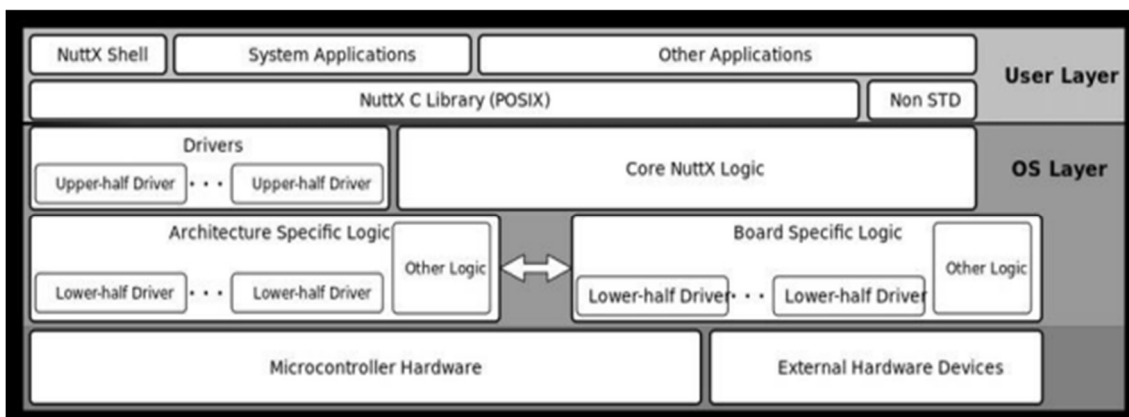
Collegano l'upper-half driver con il microcontrollore hardware reale.

Rappresentano la parte concreta dei driver.

Librerie:

L'implementazione di NuttX è organizzata in directory che contengono librerie modulari.

Questo rende il sistema scalabile ed estendibile.



Riassumendo:

Upper-half = parte generica, interfaccia verso l'utente.

Lower-half = parte specifica, interfaccia verso l'hardware.

Insieme garantiscono portabilità e modularità → lo stesso upper-half può funzionare con diversi dispositivi cambiando solo il lower-half.

Task in NuttX:

Definizione:

Un task in NuttX = equivalente di un processo.

È un'entità dinamica che rappresenta un programma in esecuzione.

Differenze con i processi classici:

Nei sistemi tradizionali, ogni processo ha un spazio di indirizzi privato.

Nei microcontrollori spesso non esiste una Memory Management Unit (MMU).

NuttX quindi opera con un flat address space (tutti i task condividono lo stesso spazio di memoria).

Dunque risulta più leggero e adatto a sistemi embedded.

Task Control Block (TCB):

Ogni task è descritto da una struttura dati detta TCB, che contiene:

- Risorse associate → variabili d'ambiente, stream, file descriptor.
- Stato del task (campo task_state).

Il TCB è il riferimento principale per la gestione dei processi in NuttX.

Identificazione:

Ogni task ha un taskID univoco.

Con il taskID → il sistema operativo può recuperare il TCB e viceversa.

Relazioni parent-child:

I task sono organizzati gerarchicamente:

- Child: eredita risorse dal padre (file stream, stdin/stdout).
- Parent: attende la terminazione dei processi figli prima di essere rimosso.

IPC in NuttX:

Signals:

Qualsiasi task o un interrupt handler può inviare un segnale a un altro task (usando il suo taskID).

Il task che riceve il segnale si “risveglia” ed esegue una specifica funzione di gestione (interrupt handler).

Meccanismo semplice e veloce → usato per eventi immediati.

POSIX Message Queue:

Ogni task può avere una coda di messaggi.

Il main del task resta in attesa di nuovi messaggi.

L’interrupt handler può inviare messaggi a questa coda quando si verifica un evento.

Quando un messaggio arriva → il task si sveglia e inizia l’elaborazione.

Utile per comunicazioni strutturate tra più task.

Semafori:

Meccanismo di mutual exclusion (mutex) → impedisce che due task accedano alla stessa risorsa contemporaneamente.

Possono anche funzionare come signaling semaphores → un task segnala ad un altro quando una risorsa è disponibile.

Esempio nello schema:

Task A chiama `sem_wait(sem)` → se la risorsa non è disponibile, resta bloccato.

Quando la risorsa è libera, fa `sem_post(sem)` → la rilascia.

Task B può quindi prendere la risorsa (`sem_wait(sem)`).

Serve per garantire coerenza e sincronizzazione.

TASK A	TASK B	TASK A	TASK B
<code>sem_init(sem, 0, 0);</code>		<code>sem_wait(sem);</code>	
<code>sem_wait(sem);</code>		<i>have access</i>	
<i>blocked</i>		<i>priority boost</i>	<code>sem_wait(sem);</code>
	<code>sem_post(sem);</code>		<i>blocked</i>
<i>_Awakens as holder _</i>		<code>sem_post(sem);</code>	
		<i>priority restored</i>	<i>have access</i>

Recap:

Signals → notifiche veloci (task → task).

Message Queue → comunicazione strutturata (task ↔ handler ↔ task).

Semafori → sincronizzazione e gestione risorse condivise.

Scheduling in NuttX:

Stati dei task:

Ogni task in NuttX appartiene ad una categoria (stato specifico):

1. **inactive_tasks:** Task inizializzati ma non ancora attivati.
2. **Readytorun:** Lista dei task pronti a partire.
 - Testa = task attivo corrente (in esecuzione).
 - Coda = task in idle (in attesa di essere schedulati).
3. **Pendingtasks:** Task pronti per partire ma bloccati temporaneamente, perché:
 - il processo in esecuzione ha disabilitato la preemption, oppure
 - hanno priorità minore rispetto al task attualmente in esecuzione.
4. **Blocked:** Task in attesa di un evento per poter proseguire.

Suddivisi in:

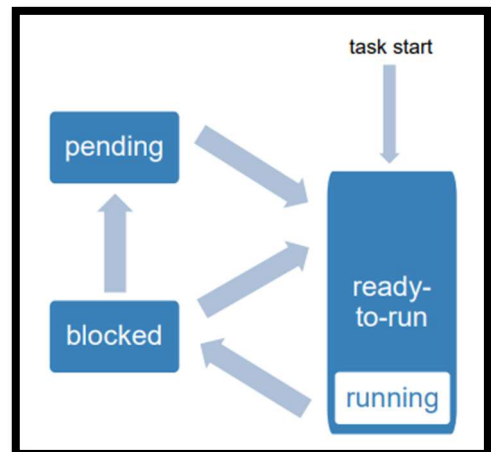
- waitingforsemaphore → in attesa di un semaforo.
- waitingforsignal → in attesa di un segnale.

In sintesi:

Un task in NuttX può essere:

- Inactive (non avviato).
- Ready-to-run (in coda o in esecuzione).
- Pending (pronto ma in attesa per priorità/preemption).
- Blocked (in attesa di risorse/eventi).

Questo modello permette allo scheduler RTOS di gestire bene la concorrenza e rispettare le deadline real-time.



File system NuttX:

Pseudo file-system:

Non è obbligatorio per il funzionamento di NuttX.

È solo una struttura dati ad albero in RAM.

Non richiede dispositivi di storage fisici (quindi può funzionare anche senza memoria secondaria).

Funzionamento:

Se abilitato, parte da uno pseudo path root /.

È estensibile → permette di montare veri file system reali partendo da questa struttura.

Simile a Linux:

- /dev → dispositivi speciali.
- /mnt → punto di mount per altri file system.

Interfaccia POSIX:

L'implementazione segue lo standard POSIX.

Funzioni principali:

- open()
- close()
- read()
- write()

Quindi la gestione dei file in NuttX è compatibile con i programmi scritti per sistemi Unix-like.

In sintesi:

NuttX può funzionare senza file system.

Usa uno pseudo file system in RAM.

Supporta mount di FS reali (utile per dispositivi esterni).

È compatibile con lo standard POSIX → portabilità del codice.

Sottosistema grafico NX:

Architettura client/server:

Funziona in modo simile a X-Window su Linux.

Client → richiede la rappresentazione degli oggetti grafici.

Server (back-end NX) → esegue il rendering e gestisce le primitive grafiche.

Questo modello separa chi richiede le operazioni (applicazioni) da chi le esegue (motore grafico).

Gestione finestre:

Le finestre sono disposte in uno spazio verticale virtuale.

Ci sono finestre attive e inattive, gestite dal server NX.

Supporto input:

API per dispositivi di puntamento (mouse, touch).

NX inoltra gli input alle callback del client.

Supporto completo anche per le tastiere.

NxWidgets:

Sottosistema per l'integrazione di oggetti grafici ("widget") all'interno di NX.

Permette di costruire interfacce grafiche interattive.

Scritto prevalentemente in C++.

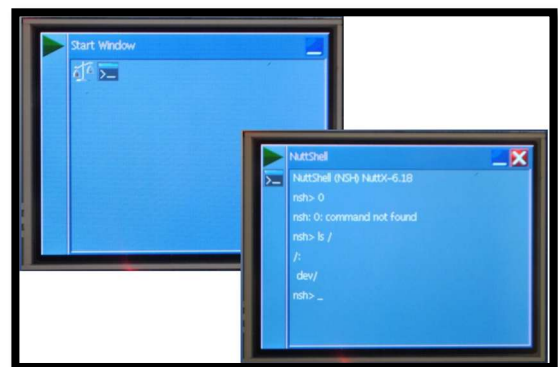
In sintesi:

NX è un sistema grafico minimale ma modulare.

Architettura client/server → separa applicazioni e rendering.

Supporta finestre, input da mouse/touch/tastiera.

Con NxWidgets si possono creare GUI semplici e leggere per sistemi embedded.



NuttShell (NSH):

Caratteristiche:

Shell di riferimento per NuttX.

Basata sulla BASH di Linux → sintassi simile.

Supporta diversi formati di comando:

- `<cmd>`
- `<cmd> > <file>` → redirectione output.
- `<cmd> >> <file>` → append su file.
- `<cmd> &` → esecuzione in background.

Variabili built-in:

PWD e OLDPWD (directory corrente e precedente).

\$? → risultato dell'esecuzione dell'ultimo comando.

Comandi principali:

pwd, cd → mostrare/cambiare directory.

ls → listare directory.

mount → montare un file system esterno nello pseudo file system.

Comandi di rete: es. ifconfig, ping.

Esecuzione di script esterni con sh.

In pratica: NSH porta in un RTOS comandi tipici di Linux.

Comportamento di default all'avvio della shell:

1. NSH crea un file system minimale:

ESEMPIO:

--init.d/

└─ rcS

2. Monta il file system in /etc.

ESEMPIO:

/dev/

└─ ram0

/etc/

└─ init.d/

└─ rcS

3. Esegue lo script prima del prompt: Questo script configura il sistema root.

ESEMPIO:

/dev/

└─ ram0

└─ ram1

/etc/

└─ init.d/

└─ rcS

/tmp/

Configurazione: I parametri principali sono definiti nel file defconfig.

In sintesi:

NSH è una mini-shell compatibile con Linux, utile per configurazione e debug.

Supporta comandi base, redirezioni, variabili, script, gestione rete.

All'avvio crea un file system minimale in RAM, esegue uno script di init (rcS), e poi mostra il prompt.

La comunicazione avviene tramite serial client.



```
arnaldo@arnaldo-X580VD: ~/Firmware_originale/Firmware
nsh> help
help usage: help [-v] [<cmd>]

[
?      cp      export  ls       mv       set      true
break  date     false   mkdir   ps       sh       umount
cat     exec     help    mount   rn       test    usleep
cd      exit     kill    mv       rndir   time

Builtin Apps:
l3gd20  airspeed_selector  fw_pos_control_l1  hmc5883
uorb    reboot            mc_att_control     sensors
ns5611  rc_update         top                serdls
load_mon ekf2              mavlink            mc_rate_control
ns4525_airspeed navigator         nsh                px4io
vtol_att_control adc               mixer              fw_att_control
gps      lsm303d           logger             perf
fmu      sercon            ntd                pwm
commander bl_update         land_detector      testMinline
mpu6000  param             dataman            mc_pos_control
nsh>
```